Listening for ships with an undersea cable

An end-to-end guide to **shipdas**: vessel detection and distance estimation from Distributed Acoustic Sensing (DAS) on a submarine fibre-optic cable. What the data is, how every step works, why each model was chosen, how to run it, and what we found.

Dataset: Marlinks-NS (Zenodo, CC BY 4.0) | Code: github.com/sriramachellu/shipdas

Hardware: laptop RTX 3060 (6 GB), 16 GB RAM

Overview

How to read this guide

1 The problem

Why cables need protecting and what DAS is

2 The dataset

Marlinks-NS: what one sample looks like and what the labels mean

3 Evaluation

Day-wise cross-validation and the metrics

4 Pipeline and code

Each script, with snippets explained line by line

5 Models

Why XGBoost, why a CNN, why a cascade of both

6 Results

Headline numbers, what each step added, where it fails

7 Run it yourself

Commands, timings, hardware notes and lessons

The problem: cables break, and we rarely see it coming

- About **99% of intercontinental internet traffic** runs through submarine fibre-optic cables.
- Most faults are caused by people, not nature: **ship anchors dragged across the seabed and fishing gear** such as trawls.
- A repair needs a specialised ship and can take weeks. Knowing that a vessel is close to the cable, and how close, lets an operator warn it before damage happens.
- Ships report their position through **AIS**, but AIS can be switched off. We want a sensor that does not depend on the ship cooperating.
- **Goal of this project:** from the cable's own vibration data, (1) decide whether a vessel is within 1 km of the cable and (2) estimate its distance.

Two tasks, one input

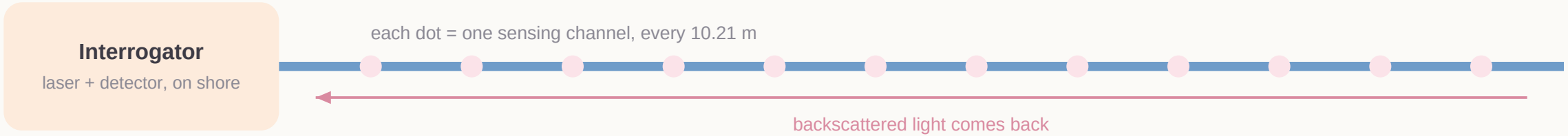
Detection (classification): is the closest vessel within 1,000 m?

Localisation (regression): how far is it, in metres?

Input for both: 10 seconds of vibration energy measured along 2.5 km of cable.

1 The problem

How a fibre becomes 2,500 microphones



- A laser pulse travels down the fibre. Tiny imperfections in the glass reflect a little light back (Rayleigh backscatter).
- When sound or vibration stretches the fibre, the phase of that reflected light changes. Timing tells us **where** along the fibre it happened.
- So one ordinary telecom fibre becomes thousands of virtual microphones. This is **Distributed Acoustic Sensing**.
- A ship's engine and propeller make low-frequency noise that reaches the seabed cable.

The dataset: Marlinks-NS

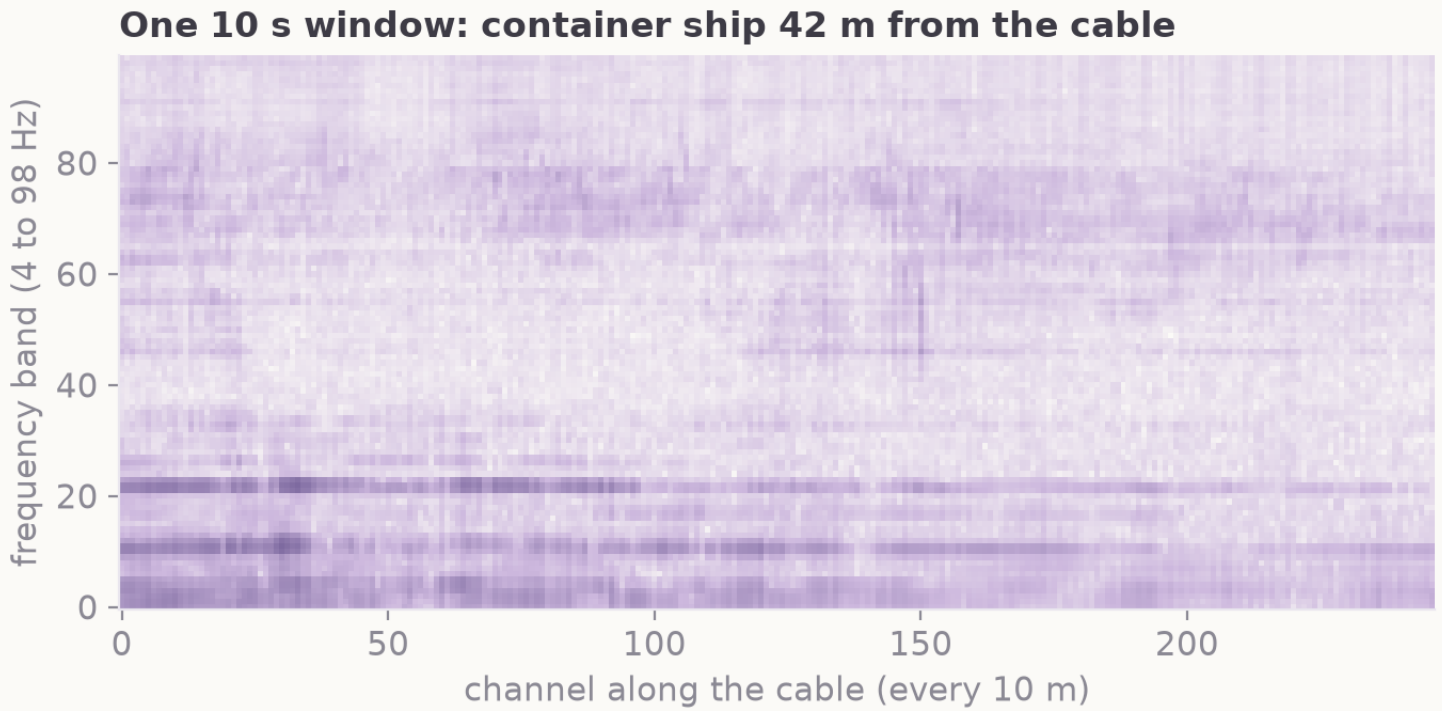
Where	28 km cable off Zeebrugge, Belgium (North Sea)
When	16 to 25 June 2023, continuous, 10 days
Instrument	Alcatel OptoDAS, sampling 3,125 Hz
Segment used	2,553 m of cable, 250 channels (247 after removing 3 noisy ones)
One sample	10 seconds, summarised as energy in 100 frequency bands, 4 to 98 Hz
Samples	74,771 windows, X has shape (74771, 250, 100)
Label y	distance in metres to the closest vessel reported by AIS
Extra	ship type, length and beam of that vessel; daily weather
Size	14.1 GB HDF5, float64; licence CC BY 4.0

Why this dataset

It is the only public DAS dataset with labelled vessel distances. The authors also published a baseline (XGBoost) and an evaluation protocol, so we can compare our numbers to theirs fairly.

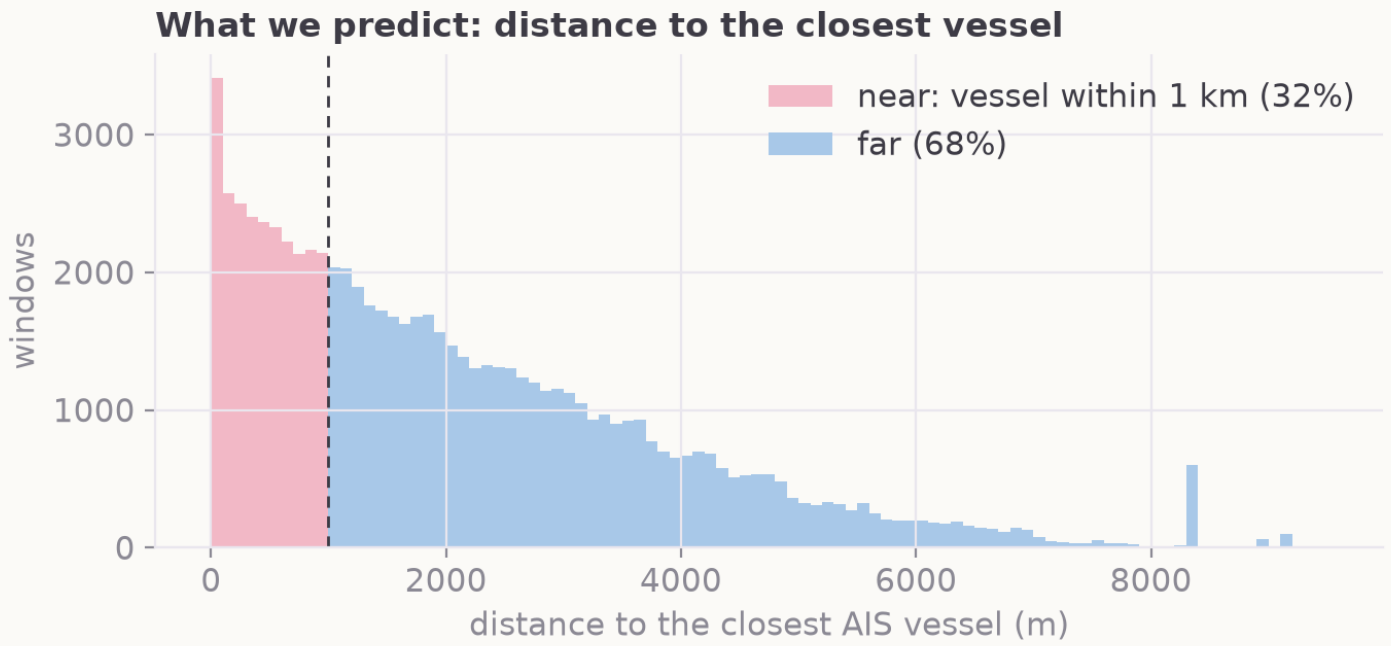
Labels come from AIS, so vessels without AIS are invisible to the labels.

What one sample looks like



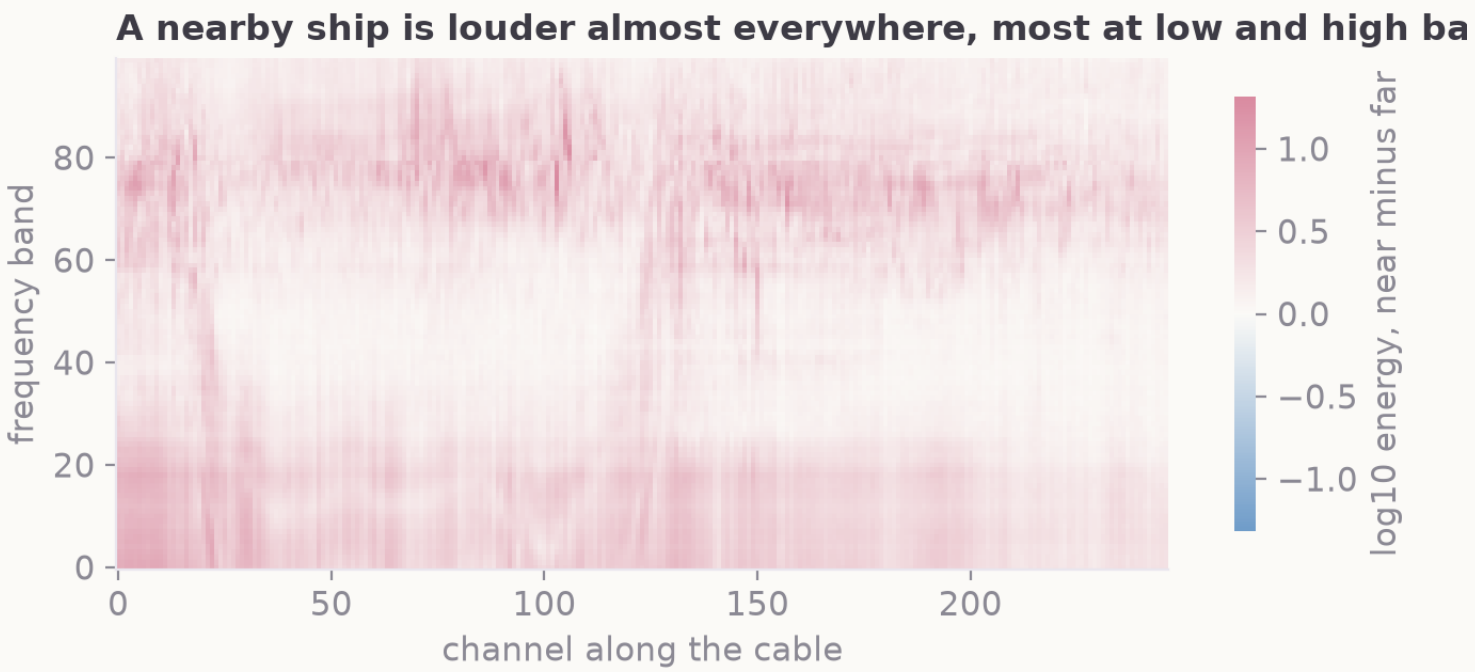
- Each window is a small image: **247 channels** (x axis, position along the cable) by **100 frequency bands** (y axis).
- Colour is the log of the vibration energy in that band at that position.
- Horizontal stripes are the ship's tonal noise: engine and propeller harmonics heard along the whole segment.
- This picture-like structure is exactly why a **CNN** (an image model) makes sense later.

What we predict



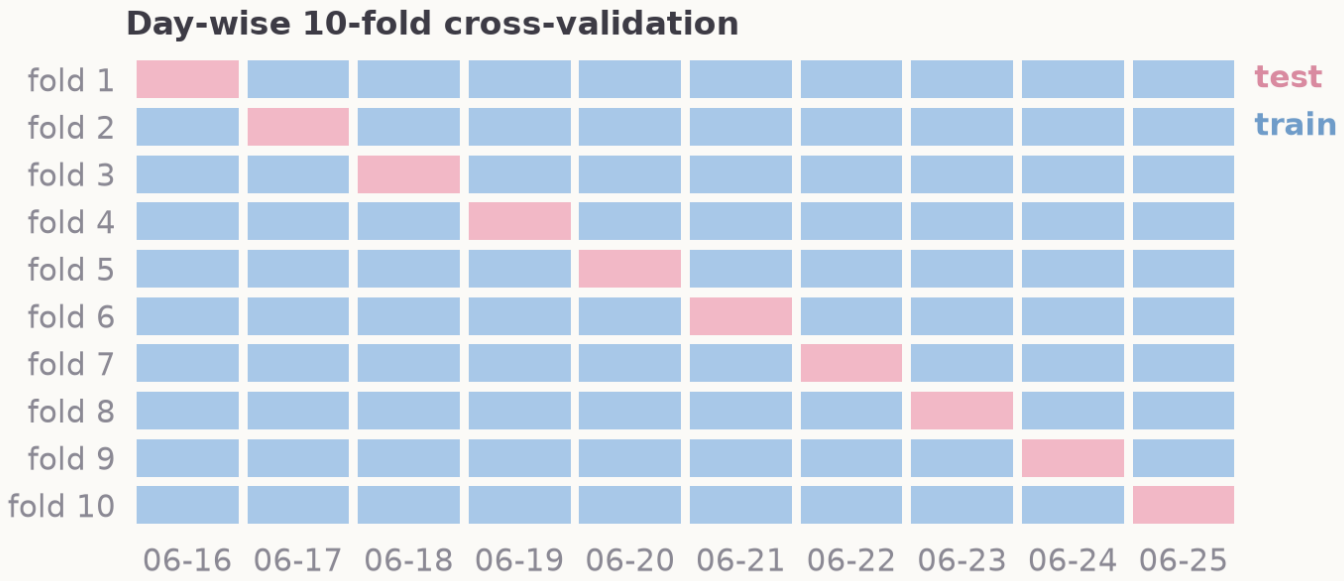
- Distance to the closest vessel ranges from 13 m to 9.1 km; the median is 1.7 km.
- Following the paper, a window is **near** when that distance is at most **1,000 m**. That gives 32% near and 68% far.
- The classes are imbalanced, so accuracy alone can mislead. We also report F1 per class, macro F1 (plain average) and weighted F1 (what the paper calls global F1).
- The noisy channels 59, 60 and 61 are already zeros in the release; we drop them.

Is there a signal? Near minus far



- We averaged 2,000 near windows and 2,000 far windows (more than 3 km) and subtracted.
- Everything is warmer: a close ship raises energy almost everywhere.
- The strongest differences are the **lowest bands** (propeller and engine) and a band near **50 to 70 Hz**.
- It is also uneven along the cable, so **where** on the cable it is loud carries information. That motivates the spatial features.

Evaluation: hold out whole days



- Windows 10 s apart are nearly identical. A random train/test split would put almost the same window in both, and the score would be far too optimistic. This is **leakage**.
- So each fold tests on one **whole day** and trains on the other nine. Ten folds, one per day, scores averaged. This is the protocol the dataset authors recommend.
- Context features only look **backwards in time**, never across a recording gap, so they would also work live.

Metrics in plain words

Precision (near)

Of the windows we flagged as near, how many really were.

Recall (near)

Of the windows that really were near, how many we caught.

F1

Harmonic mean of precision and recall, one number per class.

Macro F1

Plain average of the near F1 and the far F1. Treats both classes equally.

Weighted F1

Average weighted by class size. The paper's 'global F1'.

AUC

Chance a random near window scores higher than a random far one.

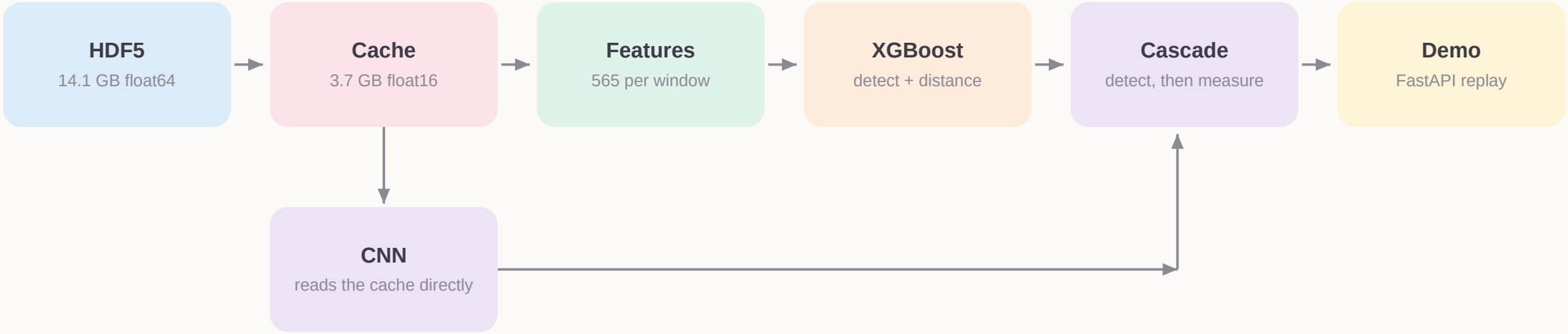
MAE

Mean absolute error of the distance, in metres. Lower is better.

Paper protocol MAE

MAE of a model trained and tested only on windows within 1 km.

The pipeline at a glance



`01_build_cache.py`	stream HDF5 into a small cache	5 min
`02_eda.py`	label stats and spectra plots	1 min
`03_train_gbm.py`	XGBoost, 10 folds, 3 models per fold	4 to 15 min on GPU
`04_train_cnn.py`	CNN, 10 folds	about 95 min
`05_report.py`	comparison table, recall by ship type	1 min
`06_cascade.py`	end-to-end evaluation	seconds

Step 1: shrink 15 GB into 3.7 GB

```
1  def build_cache(h5_path=C.RAW_H5, chunk=512):
2      C.CACHE_DIR.mkdir(parents=True, exist_ok=True)
3      with h5py.File(h5_path, "r") as f:
4          X = f["X"]
5          n = len(X)
6          offset = float(np.median(np.log10(X[:chunk][:, KEEP] + C.EPS)))
7          out = np.lib.format.open_memmap(CACHE_X, "w+", np.float16, (n, len(KEEP), C.N_BANDS))
8          lo, hi = np.inf, -np.inf
9          for s in tqdm(range(0, n, chunk), desc="caching"):
10             block = np.log10(X[s:s + chunk][:, KEEP] + C.EPS) - offset
11             lo, hi = min(lo, block.min()), max(hi, block.max())
12             out[s:s + len(block)] = block
```

1 A function with a default path; chunk = how many windows we read at a time.

3 Open the HDF5 file read-only. Nothing is loaded yet, h5py reads lazily.

6 One global offset (a median) so the stored numbers are small. Small numbers keep float16 precise.

7 open_memmap creates a .npy file on disk that we can fill like an array.

9 Loop over the file 512 windows at a time, about 100 MB each. RAM never fills.

10 Drop the noisy channels, take log10 of the energy, subtract the offset.

8, 11 Track the smallest and largest value; the uint8 GPU store needs that range.

12 Write the block into the cache. float64 to float16 is 4x smaller.

Step 2: one fold per day

```
1 def day_folds(days):
2     for d in sorted(set(days)):
3         test = days == d
4         yield d, np.flatnonzero(~test), np.flatnonzero(test)
```

Why a generator? The caller writes `for day, tr, te in day_folds(ds.days):` and gets one fold per loop, without building all ten up front.

1. days is an array with the date of every window, for example '2023-06-16'.
2. For each distinct day, in date order...
3. ...a boolean mask: True where the window belongs to the test day.
4. yield hands back the day name, the training indices (every other day) and the test indices. Used by every training script, so they all share the same 10 folds.

Step 3: features for XGBoost

```
def block_features(L):
    L = L.astype(np.float32)
    top = L.max(axis=1)
    mean = top + np.log10(np.mean(10.0 ** (L - top[:, None]), axis=1))  # stable log-mean-exp
    ch = np.linspace(0, L.shape[1], N_SEG + 1).astype(int)
    fb = np.linspace(0, L.shape[2], N_FGROUP + 1).astype(int)
    seg = [L[:, ch[i]:ch[i + 1], fb[j]:fb[j + 1]].mean(axis=(1, 2))
           for i in range(N_SEG) for j in range(N_FGROUP)]
    return {"mean": mean, "std": L.std(axis=1), "max": top, "seg": np.stack(seg, axis=1)}
```

2 Work in float32; the cache is float16.

3 Loudest channel for each band. Used twice: as a feature and for a stable average.

4 The paper's feature: average the linear energy over channels, then log. Doing it as top + log(mean(10^(L - top))) avoids overflow.

5-8 Cut the cable into 13 segments and the bands into 5 groups, and average each block: 65 numbers describing where along the cable it is loud.

9 Return mean, spread (std), max and segments: 100+100+100+65 = 365 features.

5 Models

Why XGBoost first

- **It is the paper's baseline.** Replicating it first proves our data handling is right before trying anything new. We got 89.35% accuracy against their 89.58%.
- Gradient-boosted trees are the strongest standard model for **tabular features**: fast, little tuning, robust to features on different scales.
- It trains on the GPU: 10 folds in about 4 minutes for 100 features.
- Trees can use any feature, so it tells us cheaply whether the new spatial features help (they did: +1.4 points macro F1).
- Hyper-parameters copied from the paper: learning rate 0.05, depth 10, 500 trees.

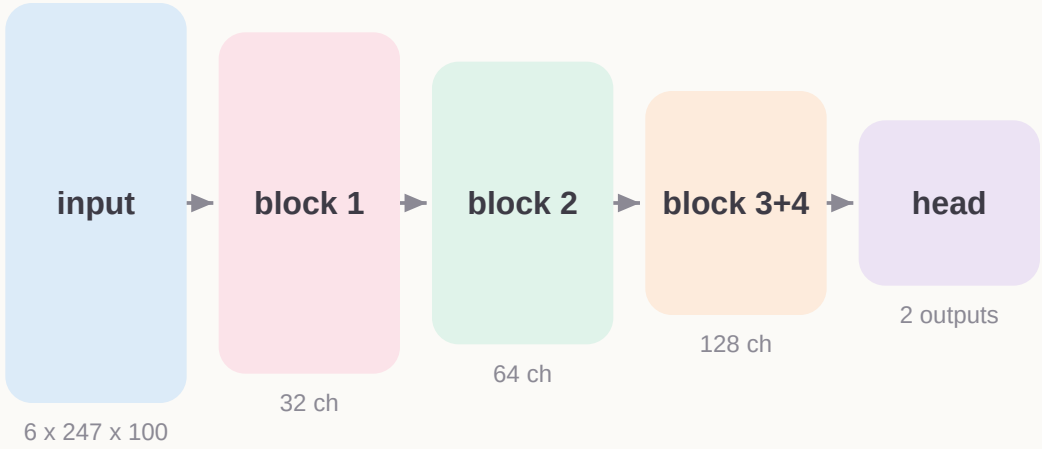
```
for day, tr, te in day_folds(ds.days):
    kw = dict(PARAMS, tree_method="hist", device=device)
    clf = xgb.XGBClassifier(**kw).fit(X[tr], y_det[tr])
    reg = xgb.XGBRegressor(**kw).fit(X[tr], y[tr])
    trn = tr[y[tr] < C.THRESHOLD_M]
    reg_n = xgb.XGBRegressor(**kw).fit(X[trn], y[trn])
```

Three models per fold: **clf** for near/far, **reg** for distance on all windows, **reg_n** for distance trained only on windows within 1 km (the paper's protocol, and the distance model in our final cascade).

5 Models

Why a CNN next

- Hand-made features throw information away. Each window is really a 247 x 100 **image**, and convolutional networks are built to read images.
- Convolutions find local patterns (a harmonic stripe, a loud stretch of cable) wherever they appear. Max pooling lets the model react to a ship at any position.
- Stacking the **5 previous windows** (50 s) as extra input planes gives the model motion: a ship approaching gets louder over time.
- One network, two outputs (multitask): a near/far logit and a distance. Sharing the features helps both.



Each block: conv 3x3, batch norm, ReLU, twice, then 2x2 max pool. The head pools the map with both average and max, then two linear layers.

Code: looking back 50 seconds, safely

```
1 def context_index(contiguous_prev, k):
2     n = len(contiguous_prev)
3     idx = np.empty((n, k + 1), dtype=np.int64)
4     idx[:, k] = np.arange(n)
5     for j in range(k - 1, -1, -1):
6         nxt = idx[:, j + 1]
7         idx[:, j] = np.where(contiguous_prev[nxt] & (nxt > 0), nxt - 1, nxt)
8     return idx
```

1 contiguous_prev[t] is True when window t starts exactly 10 s after t-1.

3 A table with k+1 columns: indices of the past windows, then the current one.

4 The last column is the current window itself.

5 Fill the columns from right to left.

6-7 Step one window back only if the recording was continuous; otherwise repeat the same window. So we never jump across a gap and never look into the future.

Code: the CNN

```
1  class DasNet(nn.Module):
2      def __init__(self, in_planes=1, width=32, dropout=0.3):
3          super().__init__()
4          w = width
5          self.features = nn.Sequential(
6              Block(in_planes, w), Block(w, 2 * w), Block(2 * w, 4 * w), Block(4 * w, 4 * w))
7          self.head = nn.Sequential(
8              nn.Linear(8 * w, 128), nn.ReLU(), nn.Dropout(dropout), nn.Linear(128, 2))
9
10     def forward(self, x):
11         h = self.features(x)
12         h = torch.cat([h.mean(dim=(2, 3)), h.amax(dim=(2, 3))], dim=1)  # average + max pooling
13         out = self.head(h)
14         return out[:, 0], out[:, 1]
```

2 in_planes = 1 + context windows; width sets the number of filters (32).

5-6 Four Blocks double the filters while each halves the image size.

7-8 The head: 256 pooled numbers to 128 hidden units, dropout 0.3, then 2 outputs.

12 Global average and global max pooling, side by side. Max catches a ship that is loud on only part of the cable.

14 Output 0 is the near/far logit, output 1 the standardised distance.

Code: one training step

```
for b in batches(tr, args.bs, rng):
    bt = torch.from_numpy(b)
    with torch.autocast("cuda", dtype=torch.float16):
        logit, d = model(inputs(b, args.gain_aug))
    loss = F.binary_cross_entropy_with_logits(logit.float(), yd[bt].to(dev))
    r = reg_loss(d.float(), yt[bt].to(dev), reduction="none") * w[bt].to(dev)
    loss = loss + r.sum() / w[bt].sum().clamp(min=1).to(dev)
    opt.zero_grad(set_to_none=True)
    scaler.scale(loss).backward()
    scaler.step(opt)
    scaler.update()
    sched.step()
    total += loss.item() * len(b)
```



8 epochs per fold, AdamW, peak learning rate 0.002. About 10 minutes per fold.

1	Shuffle the training windows and cut them into batches of 128.
3	Mixed precision (float16) makes the 6 GB laptop GPU faster and fit more.
4	inputs() gathers the 6 windows, normalises each band and adds a small random gain.
5	Binary cross-entropy for the near/far output.
6-7	Distance loss, masked by w so it can be limited to near windows if we want.
8-12	Standard step: clear gradients, backpropagate (scaled for float16), update, and move the one-cycle learning-rate schedule.

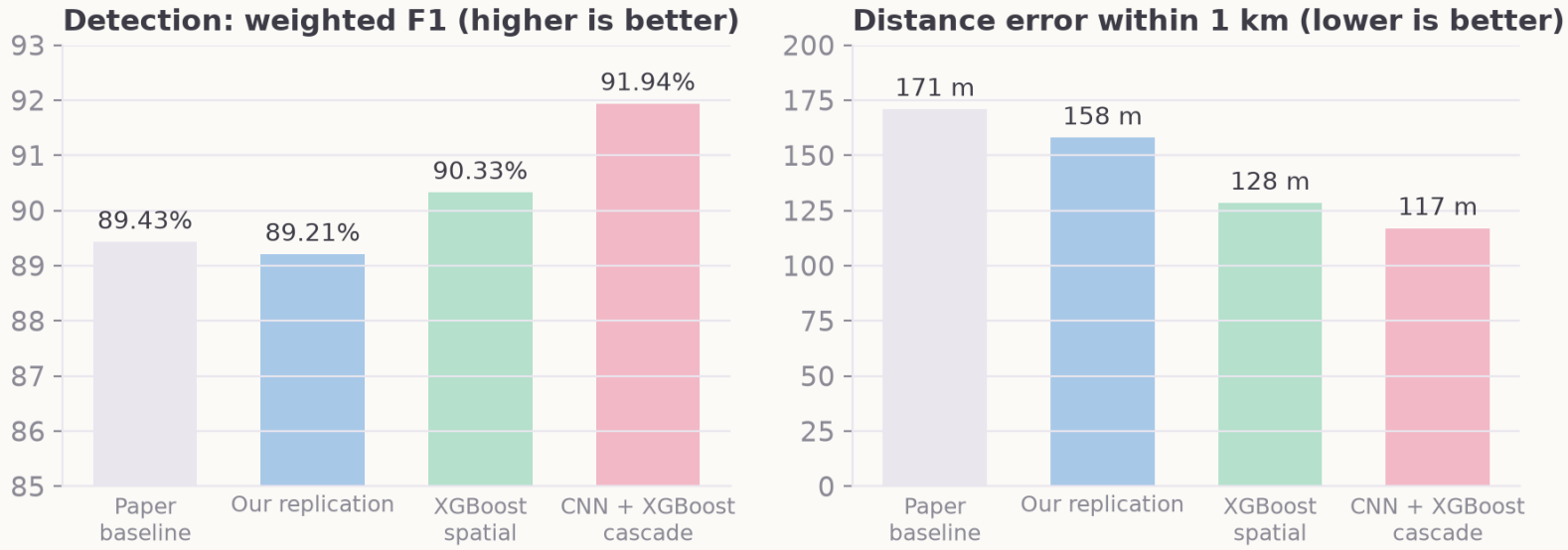
5 Models

Why a cascade of both



- The CNN was the best **detector** (90.7% macro F1) but its distance head under the paper protocol was worse than XGBoost (152 to 206 m against 129 m).
- XGBoost on spatial features was the best **distance** model within 1 km.
- So we chain them, the way a deployed system would work: detect first, then measure.
- This is also a fairer test than the paper's protocol, which assumes we already know which windows are within 1 km.

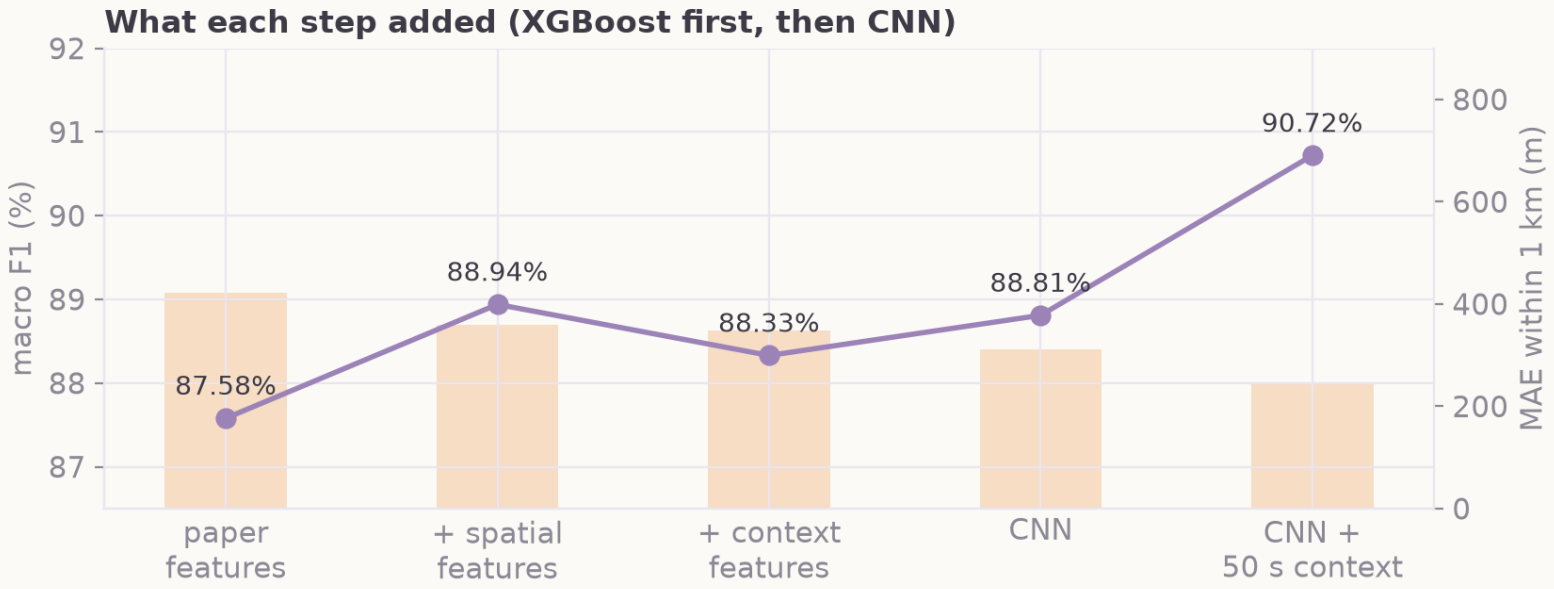
Results: better than the paper on both tasks



All numbers: mean over 10 held-out days.

- **Detection:** weighted F1 91.94% against the paper's 89.43%; macro F1 90.72%, AUC 0.963.
- **Distance, fair comparison:** 128.5 m against 171 m, same protocol.
- **End to end:** 117 m on vessels the CNN detected (median 85 m). Detected vessels are somewhat easier, so compare the 128.5 m with the paper.

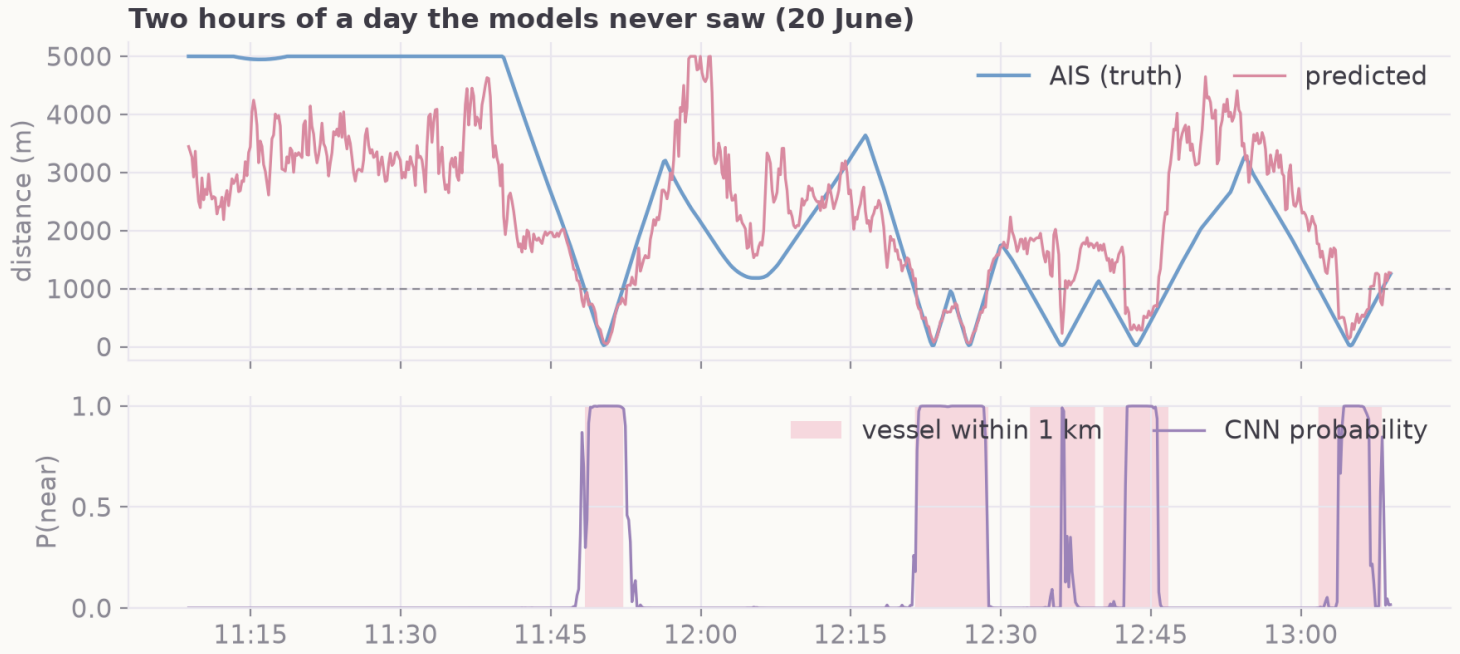
What each step bought

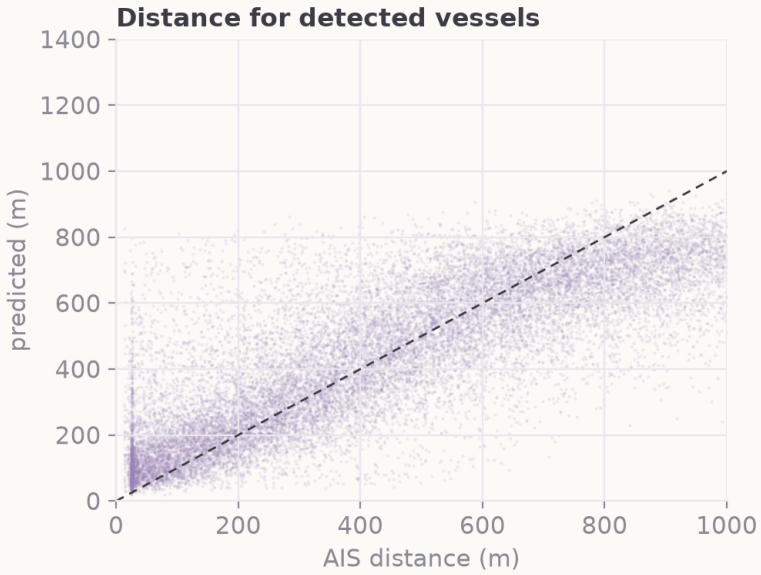


- Spatial features: +1.4 points macro F1 and 62 m less error. Where on the cable it is loud matters.
- Hand-made context features did **not** help XGBoost.
- Learned context did help the CNN: +0.9 points and 38 m less error on the screening days.
- Bars: distance error of models trained on all distances, measured within 1 km.

6 Results

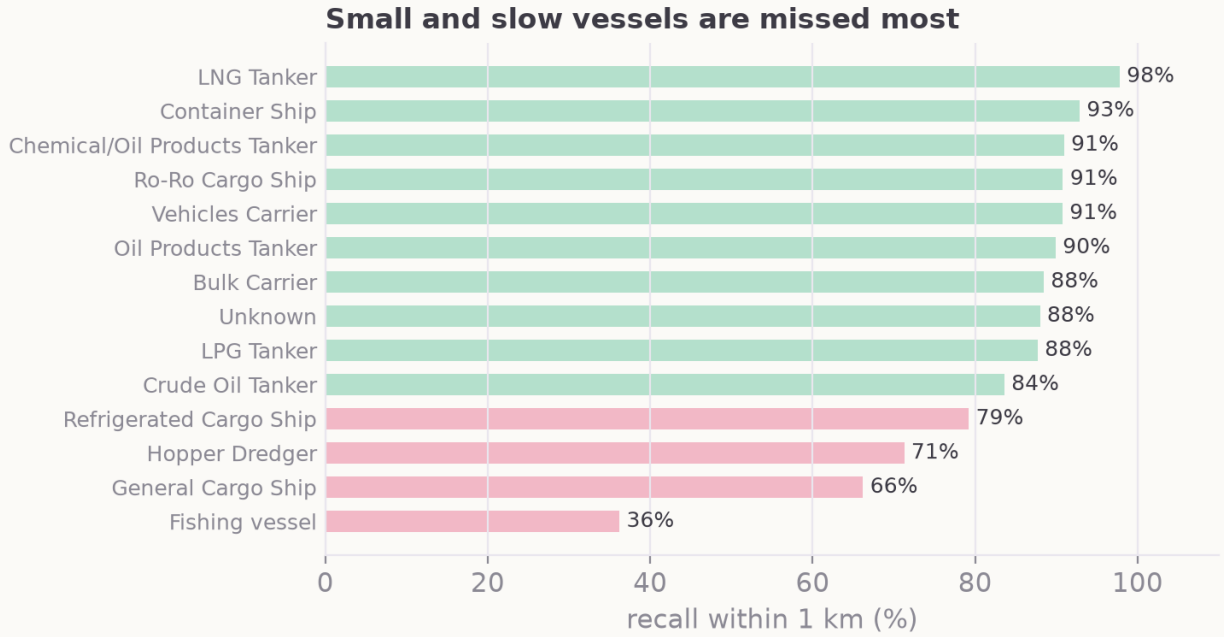
What the predictions look like

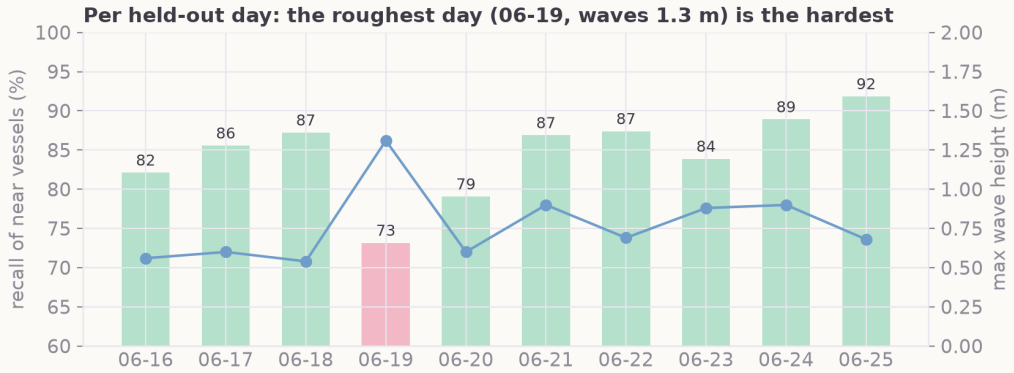




Left: two hours of a held-out day. The probability jumps up when a vessel enters the 1 km band. Right: predicted against true distance for every detected vessel.

Where it fails





Fishing vessels (36%) and dredgers (71%) are missed most, and they are real cable threats. The roughest day was the hardest, but 10 days are too few to prove a weather effect.

Run it yourself

```
uv venv .venv --python 3.12
uv pip install --python .venv/Scripts/python.exe torch \
    --index-url https://download.pytorch.org/whl/cu126
uv pip install --python .venv/Scripts/python.exe -e ".[dev]"

# put the Zenodo files in data/raw/, then:
python scripts/01_build_cache.py
python scripts/02_eda.py
python scripts/03_train_gbm.py --sets paper spatial --tag paperhp
python scripts/03_train_gbm.py --sets spatial --tag cascade
python scripts/04_train_cnn.py --epochs 8 --context 5 --gain-aug 0.1 \
    --save-model --tag cnn_c5
python scripts/06_cascade.py
python scripts/05_report.py
python -m shipdas.serve --day 2023-06-20    # open http://127.0.0.1:8765
python -m pytest -q
```

Before you start

Download from Zenodo record 15611778: the 14.1 GB .h5 file and misc.zip. Check the MD5: 407ad62bb8eb02f472cda329d0f31750.

Python 3.12, an NVIDIA GPU with CUDA 12, about 25 GB free disk.

Engineering notes from a 16 GB laptop

RAM, not GPU, was the limit

The float64 file is bigger than RAM, so the cache is built in chunks. Two runs at once crashed with out-of-memory.

Data on the GPU as uint8

The --gpu-data flag stores the dataset on the GPU in 8 bits (1.85 GB). Same accuracy, frees 3.7 GB of RAM.

Parallel download

Zenodo gave 2 MB/s per connection. Eight range requests in parallel reached about 20 MB/s; the MD5 matched.

Leave GPU headroom

One CNN run crashed with a cuDNN error when a browser took GPU memory; the rerun was fine.

Limitations and next steps

- **One site:** 2.5 km of one cable, one sea, ten summer days. Another cable or season will need re-validation.
- **AIS labels:** vessels without AIS are missing, and positions are interpolated.
- **Mild selection bias:** CNN settings were picked on two of the ten days, which are also in the average. The gain held on the other eight.
- **Next:** class weighting or more data for small vessels; a single model that is good at both tasks; test on the Pacific OOI cable data; stream from a live interrogator.

Sources

References

- Ramirez-Torres et al., Marlinks-NS DAS dataset, Zenodo, doi:10.5281/zenodo.15611778 (CC BY 4.0).
- Ramirez-Torres et al., Vessel detection and localization using DAS in submarine optical fiber cables, IEEE JSTARS 2026, doi:10.1109/JSTARS.2026.3716768.
- Ramirez-Torres et al., A DAS dataset for vessel detection and localization in submarine cable protection, arXiv:2607.28306.
- Companion code: github.com/UAH-PSI/das-vessel-detection (GPL-3.0).
- XGBoost: Chen and Guestrin, KDD 2016. PyTorch: Paszke et al., NeurIPS 2019.